Order Is a Superpower

Week 8 — Searching & the Elementary Sorts

Dr. Auqib Hamid Lone

PCC CS-301: Data Structures
GCET, Ganderbal

Week 8

Concepts That Click

Today's Four Questions

1. Unordered or sorted — how much does order actually buy a search?
2. What precondition does binary search silently assume, and what happens when it is violated?
3. Can we search for a value we never stored — the integer square root of n?
4. Insertion, selection, bubble, counting — which sort does each input reward, and at what cost?

Plan

Write the search contract. Price linear versus binary. Search a value that is not in any array. Then sort: four algorithms, their traces, and their honest bills.

Handoff: What Week 7 Left Open

- Week 7 finished the chain family: the doubly linked list (`p->prev`, the two-link invariant), circular rings that stop at `head` instead of `NULL`, the stack and queue rebuilt on nodes, and polynomials as ordered $(\text{coeff}, \text{exp})$ lists.
- It closed on one promise: "Searching — linear versus binary, and the elementary sorts that put data in the order binary search needs."
- The deficiency it left: every lookup so far has been a *walk*. Our symbol table — a list since Week 6 — answers a lookup by scanning from `head`, $O(n)$ worst case.
- New symbols today: the search bounds `lo`, `hi`, `mid`; the properties **stable** and **in-place**; and $T(n)$ for the four elementary sorts.

This week's job

Show that *order* turns an $O(n)$ scan into an $O(\log n)$ probe — then learn the cheapest ways to create that order.

The Calculator Looks Up an Identifier

- The tokenizer has just produced the identifier `x`. The symbol table must answer: *is `x` declared, and where?*
- Held as a linked list (Week 6), each lookup scans from `head` until it matches or hits `NULL`: $O(n)$ comparisons. For a program with $2{,}000$ identifiers, that is up to $2{,}000$ comparisons *per token*.
- If the same entries are kept *sorted by name*, a binary search answers in about $\log_2 2{,}000 \approx 11$ comparisons — roughly $180\times$ fewer.
- Two questions follow: how much does sorted order speed up search, and how do we get data sorted without paying more than we save?

Running application (weeks 3–10): one calculator, rebuilt each week. Week 8 asks what its identifier table should do to look things up fast.

Every lookup so far has been a walk. Today: what does order buy?

Concepts Click

Section 1

Linear and Binary Search

Concepts That Click

Searching a Sequence: The Contract

- Finding a name in an *unordered* pile means checking one item at a time. Finding an entry in a *sorted* directory lets you jump to the middle and discard half at a glance. The difference is not cleverness — it is a precondition.

Definition — the search operation (Karumanchi Ch. 11, pp.547–589 PDF; Wirth Sec. 1.9.1, p. 34) [,]

`search(A, n, key)` returns an index i with `A[i] == key`, or -1 if `key` does not occur. **Linear search** makes no assumption about the array. **Binary search** *requires* the array to be sorted in nondecreasing order — this precondition is part of the contract, not an optional extra.

- Worked example: in `[7, 3, 9, 1, 5, 8]`, searching 9 feels the way linear search works — scan left to right. In the sorted `[1, 3, 5, 7, 8, 9]`, the same target can be found by halving.

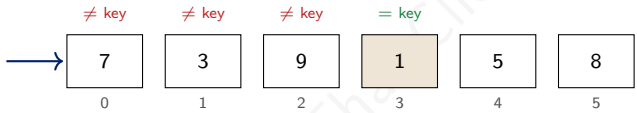
Linear Search: Scan Until You Find It

Linear search in C (Karumanchi Ch. 11, pp.547–589 PDF)

```
int lsearch(int A[], int n, int key)
  for (int i = 0; i < n; i++)
    if (A[i] == key) return i;   // hit: return the index
  return -1;                     // scanned the whole array: miss
```

- Every element is tested at most once. The loop touches `A[0]`, then `A[1]`, and so on, in index order — no assumption about the values is used.
- Cost, by case:
  - best: the first element matches — 1 comparison.
  - worst: absent, or last — n comparisons.
  - average (a uniformly random hit): $(n+1)/2$ comparisons.
- All three are $\Theta(n)$: linear search is a linear scan, and no ordering can change that.

Linear Search, Drawn



4 comparisons, then return 3

- The walk is unavoidable without order: nothing in the array tells us 1 cannot still be anywhere ahead, so each cell must be checked.

Concepts That Click

The Sorted Precondition — and What Its Absence Costs

- Binary search compares the target with the *middle* element, then keeps only the half that could contain it. That logic is sound *only if* everything left of the middle is smaller and everything right is larger — exactly the sorted invariant.

Precondition (Karumanchi Ch. 11, pp.547–589 PDF)

Binary search on $A[0..n-1]$ requires $A[0] \le A[1] \le \cdots \le A[n-1]$. Applied to unsorted data it does not error — it returns a *plausible wrong answer*, silently discarding the half that really held the key.

- Failure trace: search 1 in the unsorted `[7, 3, 9, 1, 5, 8]`. `mid = 2` (value 9); since $1 < 9$ the algorithm discards indices $3..5$ — including the 1 at index 3. It reports a miss.
- **Lesson**: the precondition is a *contract*, not a hint. Binary search is only as correct as the order it is handed.

Binary Search: Halve the Range

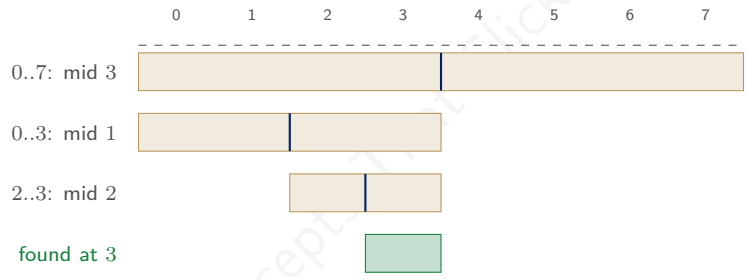
Binary search in C (Wirth Sec. 1.9.2, pp.34–36; Karumanchi Ch. 11, pp.547–589 PDF) [,]

```
int bsearch(int A[], int n, int key)
  int lo = 0, hi = n - 1;                // current [lo, hi], inclusive
  while (lo <= hi)
    int mid = lo + (hi - lo) / 2;        // avoids lo+hi overflow
    if (A[mid] == key) return mid;       // hit
    else if (A[mid] < key) lo = mid + 1; // key is in the right half
    else hi = mid - 1;                   // key is in the left half

  return -1;                             // range empty: miss
```

- Each iteration either stops or halves the live range $[\texttt{lo}, \texttt{hi}]$. Writing `lo + (hi - lo) / 2` rather than `(lo + hi) / 2` is not cosmetic: for large indices the sum can overflow `int` — Week 1's caution about C arithmetic in disguise.

Binary Search, Drawn: The Range Shrinks



each step halves the range: blue rule = `mid`

Concept_Institute

Why Binary Search Is $O(\log n)$

- Let $T(n)$ be the worst-case number of comparisons for a range of size n. One comparison, then a range of size at most $\lceil n/2 \rceil$ — so

$$T(n) = T(\lceil n/2 \rceil) + 1, \qquad T(1) = 1.$$

- Unrolling: after k halvings the range is $n/2^k$, which reaches 1 when $k = \log_2 n$. Hence $T(n) = \Theta(\log n)$ worst and average case, and $O(1)$ best case (the target is the middle at once).

n	10	100	1,000	1,000,000
comparisons $\approx \log_2 n$	4	7	10	20

Sedgewick classifies binary search as the canonical *logarithmic* order of growth ([4], order-of-growth table, p. 187; cost model, Sec. 1.4, p. 172). Doubling n adds just one comparison.

Binary Search Variants: First and Last Occurrence

Lower bound — first index with `A[i] >= key` (the upper bound is the same with $>$; Karumanchi Ch. 11, pp.547–589 PDF)

```
int lowerBound(int A[], int n, int key)    // first i with A[i] >= key
  int lo = 0, hi = n;                        // half-open [lo, hi)
  while (lo < hi)
    int mid = lo + (hi - lo) / 2;
    if (A[mid] < key) lo = mid + 1;          // discard [lo, mid]
    else hi = mid;                           // keep mid as a candidate

  return lo;                                 // n if every element < key
```

- The occurrence count is `upperBound − lowerBound`, in $O(\log n)$ — no scan. Keeping `mid` as a candidate pinpoints the *first* match.

Check Your Prediction: The Unsorted Array

The array `[7, 3, 9, 1, 5, 8]` is unsorted. You run binary search for the key 3. What does it return?

A: index 1 B: a wrong index or -1 C: a runtime error

1. A — only if the search kept the half that still contains 3; it does not.
2. B — `mid = 2` (value 9); $3 < 9$, so it discards indices $3..5$ and continues left; the range collapses to $\{0\}$, the value 7 there is > 3, and the search returns -1 — silently losing the 3 that sits at index 1.
3. C — the code compiles and runs; nothing checks the precondition.

Correct: B. Binary search on unsorted data never errors — it returns a plausible-looking wrong answer, which is worse. Sort first.

Search Cost, Side by Side

algorithm	precondition	best	worst	space
linear search	none	$O(1)$	$\Theta(n)$	$O(1)$
binary search (iterative)	sorted	$O(1)$	$\Theta(\log n)$	$O(1)$

- Binary search's $\Theta(\log n)$ worst case looks strictly better — and it is, *once you own sorted data*. The catch is the precondition: it costs something to establish, and it must be maintained after every insertion.
- The rest of this deck is about that cost. And before we leave search: two problems where binary search finds an answer that is *not* stored anywhere in the array at all.

Content That Rocks

Order halves the search. But binary search is not only for arrays of data.

Concepts to Click

Section 2

Two Applications of Binary Search

Concepts That Click

Application 1: The Integer Square Root of n

- Suppose we want $\lfloor \sqrt{n} \rfloor$ — the largest integer k with $k^2 \le n$ — using only integer arithmetic, no floating point. The answer k is not stored in any array.
- The trick: search the *answer space* $0..n$ instead of an array. Define a predicate $P(k) = (k^2 \le n)$. As k grows, P is *true* up to a point and *false* after it — a single true→false boundary.
- A monotone predicate is exactly what binary search needs: find the largest k with $P(k)$ true, and that k is $\lfloor \sqrt{n} \rfloor$.

The pattern: binary search on a monotone predicate

Whenever "is k big enough?" is a yes–no question with one flip point, binary search finds that point in $O(\log(\text{range}))$ probes — even when no element of an array is involved.

Integer Square Root: Code and Predicate

Floor of the square root by binary search on the answer (standard treatment; Karumanchi Ch. 11, pp.547–589 PDF)

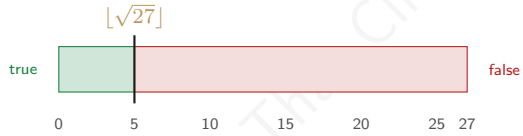
```
int isqrt(int n)                          // floor(sqrt(n)), n >= 0
  int lo = 0, hi = n, ans = 0;
  while (lo <= hi)
    int mid = lo + (hi - lo) / 2;
    if ((long long) mid * mid <= n)       // P(mid) true
      ans = mid; lo = mid + 1;             // mid is feasible; push right
     else hi = mid - 1;                   // P(mid) false; push left

  return ans;                              // largest feasible k
```

- The `(long long)` cast widens *before* the multiply: `mid` can reach n, and n^2 overflows `int` — and even a 32-bit `long` on Windows (LLP64); Week 1's arithmetic caution again.
- Cost: the range $0..n$ halves each step: $O(\log n)$ probes, $O(1)$ space — no array, no floating point.

The Square-Root Search Space, Drawn

one true→false boundary; binary search finds it in $O(\log 27)$ probes



k	0	1	2	3	4	5	6	7
k^2	0	1	4	9	16	25	36	49
$P(k)$	T	T	T	T	T	T	F	F

$25 \le 27 < 36$, so the boundary sits at $k = 5$.

Concepts Tech Clerk

Application 2: The Longest Common Prefix

- The calculator's autocomplete must find the longest prefix shared by all identifiers starting with a typed stem. For "flower", "flow", "flight" the shared prefix is "fl".
- Two facts make this a binary-search problem:
  1. If the strings are *sorted*, the longest common prefix of the whole set equals the common prefix of the first and the last string — only two strings need comparing.
  2. The length L of the common prefix of two strings is found by binary search: the predicate "the first L characters are equal" is true for small L and false beyond the answer — again a single true→false boundary.
- So: sort the set once ($O(n \log n \cdot m)$ for strings of length up to m), then answer each prefix query in $O(m)$ using first and last — no scan over all n strings.

Longest Common Prefix: Code

LCP of two strings by binary search on the length (standard treatment; Karumanchi Ch. 15, pp.640–684 PDF, string search)

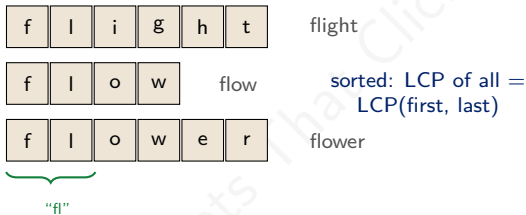
```
int lcp2(const char *s, const char *t)    // common prefix of s, t
  int lo = 0, hi = min(strlen(s), strlen(t)), ans = 0;
  while (lo <= hi)
    int L = lo + (hi - lo) / 2;
    if (prefixEq(s, t, L))  ans = L; lo = L + 1;   // P(L) true
    else hi = L - 1;                                  // P(L) false

  return ans;

// For a SORTED set: LCP of all = lcp2(first, last).
```

- `prefixEq(s, t, L)` costs $O(L)$; trying $O(\log m)$ lengths makes the query $O(m \log m)$ — or $O(m)$ if each check is incremental.
- Sorted-set fact: LCP of all = LCP(first, last) — no $O(n)$ scan.

Longest Common Prefix, Drawn



- Checking all three strings against "fl" is unnecessary once they are sorted: only `flight` and `flower` need comparing — their common prefix *is* the set's.

Concepts That Click

Check Your Prediction: Prefix Length

For the sorted set `["flight", "flow", "flower"]`, the predicate "all share a prefix of length L" is tested. For which L is it first *false*?

A: $L = 1$ B: $L = 2$ C: $L = 3$

1. A — length 1 is "f", shared by all three: true.
2. B — length 2 is "fl", shared by all three — still true, so the boundary is *after* this.
3. C — length 3 would require "fli"/"flo" to agree; "flight" has "i" where "flow" has "o". First false at $L = 3$.

Correct: C. The answer (longest common length) is 2, the largest L that is still true — binary search returns L_{true} and the prefix is its first L characters.

Two answers found by halving. Now: how do we create the order they need?

Concepts Click

Section 3

Elementary Sorting

Concepts That Click

Why Sort at All?

- Binary search is the reward; sorting is the price we pay for it. If the identifier table is sorted once and read many times, that price is amortized away.
- Sorting is not only for search. It puts "equal" things next to each other — which is how we deduplicate, find duplicates, group records, and compute medians. The calculator's symbol table, its autocomplete set, and its report of repeated declarations all begin with a sort.
- Four algorithms, introduced not as a list to memorize but as four answers to "what is the cheapest way, given *this* input?": insertion, selection, bubble, and counting sort.

A look at all four, in one line each

Insertion: grow a sorted prefix by inserting the next element. Selection: repeatedly pick the minimum. Bubble: swap adjacent out-of-order pairs. Counting: use the key values themselves as array indices.

The Three Comparison Sorts, at a Glance

sort	idea	signature
insertion	keep $A[0..i-1]$ sorted; insert $A[i]$ into place	adaptive: near-sorted input is fast
selection	find the minimum of the unsorted part; swap it forward	exactly $n-1$ swaps, always $\Theta(n^2)$ comparisons
bubble	swap adjacent inversions; the largest "bubbles" to the end	can stop early when a pass makes no swap

- All three sort *by comparing pairs* of elements, so they work on any comparable type. Counting sort, next, breaks that property and gets linear time — but only for keys from a small integer range.
- The first new vocabulary: a sort is **stable** if equal keys keep their original relative order, and **in-place** if it uses $O(1)$ auxiliary space. Both will decide real choices.

Reference treatment: Knuth, *TAOCP* Vol. 3 [3]; Horowitz & Sahni Ch. 7.

Insertion Sort: Grow a Sorted Prefix

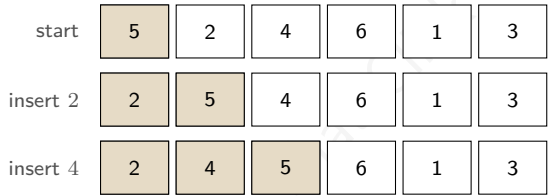
Insertion sort in C (Wirth Sec. 2.2.1, p. 46; Karumanchi Ch. 10, pp.504–546 PDF) [,]

```
void insertionSort(int A[], int n)
  for (int i = 1; i < n; i++)
    int key = A[i], j = i - 1;          // key is the element to place
    while (j >= 0 && A[j] > key)        // shift larger elements right
      A[j + 1] = A[j];
      j--;

    A[j + 1] = key;                     // drop key into its slot
```

- Invariant: before iteration i, the subarray $A[0..i-1]$ is sorted. The outer loop extends it by one; the inner loop shifts until `key` finds its place.
- Worked example: sorting `[5, 2, 4, 6, 1, 3]` — traced on the next slide.

Insertion Sort, Drawn



- Each new key is slid left past the elements greater than it. When the prefix is already sorted, the inner loop runs *once* — which is the whole point of an adaptive sort.

Insertion Sort: The Bill

- Count the shifts (the inner-loop body). Outer loop runs $i = 1 \ldots n-1$, so there are $n-1$ insertions.

case	comparisons / shifts	example input
best	$n-1$ (each key already in place)	already sorted: $O(n)$
worst	$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$	reverse sorted: $\Theta(n^2)$
average	$\Theta(n^2)$	random order

- Adaptive: cost falls to $O(n)$ on nearly-sorted input — the only one of the three comparison sorts with a linear best case.
- Stable (equal keys stay in order, because we shift only on `A[j] > key`, not $\geq$) and in-place ($O(1)$ extra space). These two properties are why insertion sort is the base case inside efficient sorts in Week 9.

Concepts That Click

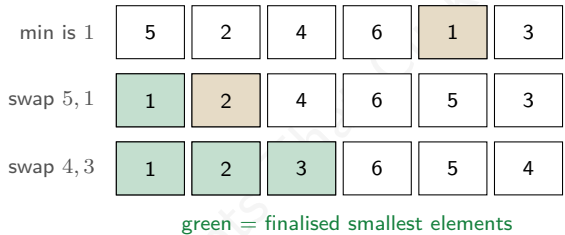
Selection Sort: Repeatedly Pick the Minimum

Selection sort in C (Wirth Sec. 2.2.2, p. 48; Karumanchi Ch. 10, pp.504–546 PDF) [,]

```
void selectionSort(int A[], int n)
  for (int i = 0; i < n - 1; i++)
    int m = i;                          // index of the minimum so far
    for (int j = i + 1; j < n; j++)
      if (A[j] < A[m]) m = j;           // find the smallest in A[i..n-1]
    swap(&A[i], &A[m]);                 // put it at the front of the rest
```

- Invariant: after iteration i, the subarray $A[0..i]$ holds the $i+1$ smallest elements, in final position. The scan finds the next minimum; one swap moves it into place.
- Note the scan always covers the whole unsorted remainder — regardless of whether the input is already sorted.

Selection Sort, Drawn



- Row 1: the scan over the whole array finds 1 (index 4); swap it with index 0. Row 2: the scan now finds 2; it is already in place. Row 3: finds 3; swap with 4.

Selection Sort: The Bill (and a Warning)

- The inner loop comparisons do not depend on the data at all: $\sum_{i=0}^{n-2}(n-1-i) = \dfrac{n(n-1)}{2}$ — every input.

case	comparisons	swaps
best	$\Theta(n^2)$	0 (already sorted)
worst	$\Theta(n^2)$	$n-1$

- Not adaptive: a sorted input is scanned just as hard as a reversed one — always $\Theta(n^2)$ comparisons. Its one virtue is the minimum number of *writes* ($n-1$ swaps), useful when moving data is far costlier than comparing it.
- Not stable: the long-range swap can leap an equal key over its twin. (Trace `[2, 2, 1]` to see the two 2s change order.)
- In-place: $O(1)$ auxiliary space.

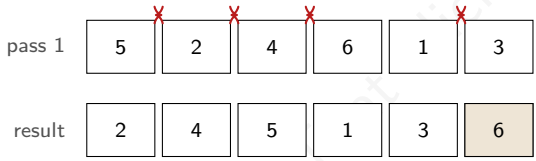
Bubble Sort: Swap Neighbours, Bubble the Max Up

Bubble sort with an early exit in C (Wirth Sec. 2.2.3, p. 50; Karumanchi Ch. 10, pp.504–546 PDF) [,]

```
void bubbleSort(int A[], int n)
  for (int i = 0; i < n - 1; i++)
    int swapped = 0;
    for (int j = 0; j < n - 1 - i; j++)
      if (A[j] > A[j + 1])  swap(&A[j], &A[j + 1]); swapped = 1;
    if (!swapped) break;     // a clean pass: already sorted
```

- Invariant: after pass i, the largest $i + 1$ elements are final at the right end, so each pass can stop i slots earlier.
- The `swapped` flag is the practical touch: a pass with no swap proves the array is sorted, so we stop.

Bubble Sort, Drawn



6 bubbles to its final slot; the next pass stops one slot sooner

- Comparing neighbours (`A[j]`, `A[j+1]`) and swapping when out of order moves the maximum rightward one step per comparison, so after one full pass it rests at the end.

Content_Not_Risk

Bubble Sort: The Bill

case	comparisons	note
best	$n-1$	sorted input: pass 1 swaps nothing, early exit
worst	$\frac{n(n-1)}{2}$	reverse sorted: $\Theta(n^2)$
average	$\Theta(n^2)$	random order

- Adaptive only *with* the early-exit flag; the naive version is $\Theta(n^2)$ on every input, including sorted data.
- Stable: equal adjacent keys are never swapped, because we swap only on strict $>$ — their relative order is preserved.
- In-place. Comparison count equals insertion sort's on average, but bubble does far more *swaps*; insertion sort is the better practical choice among the three.

All three comparison sorts hit the same $\Theta(n^2)$ wall. Week 9 asks whether any comparison sort can do better — and shows counting sort, next, sidestepping comparisons entirely.

Counting Sort: Sort Without Comparing

- All three sorts above compare keys pairwise. But if the keys are small non-negative integers, the key *is already an array index* — so it can be counted, not compared.
- Counting sort tallies how many times each value occurs, converts those counts to positions with a running total, and scatters each element to its slot. It never asks "is $A[i] < A[j]$?"
- The precondition: keys lie in a known small range $0..k-1$. The trade: linear time, but $\Theta(n+k)$ extra space — and it is only correct for keys that fit that range.

Where the calculator uses it

Counting the frequency of each byte or token class, and bucketing identifiers by length, are both one-pass counting sorts — no comparisons at all.

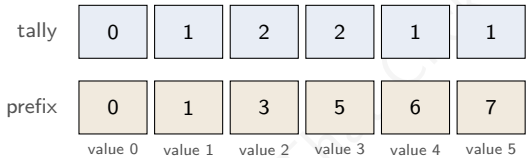
Counting Sort: Code

Counting sort with a stable scatter (Karumanchi Ch. 10, pp.504–546 PDF; Horowitz & Sahni Ch. 7, standard treatment) [,]

```
void countingSort(int A[], int n, int k)    // keys in 0..k-1
  int *count = calloc(k, sizeof(int));       // zero-initialised tally
  for (int i = 0; i < n; i++) count[A[i]]++;
  for (int v = 1; v < k; v++) count[v] += count[v - 1];  // prefix sums
  int *out = malloc(n * sizeof(int));
  for (int i = n - 1; i >= 0; i--)           // reverse pass => stable
    out[--count[A[i]]] = A[i];
  for (int i = 0; i < n; i++) A[i] = out[i];
  free(out); free(count);                    // Week 1: every malloc freed
```

- `calloc(k, ...)` zeroes the tally, so no counting slot is read before it is written. The reverse pass makes equal keys land in their original relative order — that is what makes this sort stable.

Counting Sort, Drawn: From Tallies to Positions



prefix sum = where the next occurrence of that value goes

- $A = [4, 2, 2, 5, 3, 3, 1]$: value 2 occurs twice, value 3 twice. The prefix row says the 1st slot belongs to value 0, the slots 1–2 to value 2, and so on. Scattering in reverse fills them right-to-left, preserving equal keys' order.

Counting Sort: The Bill

phase	work	cost
tally	one pass over A	$\Theta(n)$
prefix sums	one pass over $0..k-1$	$\Theta(k)$
scatter	one reverse pass over A	$\Theta(n)$
total		$\Theta(n+k)$ time, $\Theta(n+k)$ space

- Linear time when $k = O(n)$. But watch the range: if keys span a huge range (say $0..10^9$), the tally array is enormous and counting sort is worse than useless — this is why it is restricted to small integer keys.
- Stable (with the reverse scatter), not in-place ($\Theta(n+k)$ auxiliary). It sorts *without comparing keys* — so the $\Theta(n \log n)$ comparison lower bound of Week 9 does not apply to it.

The Four Sorts, Compared

sort	best	worst	space	stable	in-place
insertion	$O(n)$	$\Theta(n^2)$	$O(1)$	yes	yes
selection	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	no	yes
bubble	$O(n)$	$\Theta(n^2)$	$O(1)$	yes	yes
counting	$\Theta(n+k)$	$\Theta(n+k)$	$O(n+k)$	yes	no

- Read the table as a workload guide, not a ranking:
  - **nearly sorted** input $\rightarrow$ insertion sort: its $O(n)$ best case is the whole point.
  - **small integer keys** in a known range $\rightarrow$ counting sort: linear, but pay the $O(n+k)$ space.
  - **minimal writes** matters more than comparisons $\rightarrow$ selection sort: exactly $n-1$ swaps.
- No single winner — every row is right for *some* input. The skill is stating the workload, then choosing.

Concepts That Click

Check Your Prediction: Which Sort?

Two inputs. Which elementary sort fits each best? (i) an array that is already almost sorted (ii) 10^6 ages in $0..120$

1. For (i), **insertion sort**: nearly every key is already in place, so the inner loop rarely shifts — it runs in $O(n)$.
2. For (ii), **counting sort**: keys are integers in a tiny range, so $k = 121 \ll n$; the sort is $\Theta(n + k) \approx \Theta(n)$.
3. Selection or bubble would be $\Theta(n^2)$ on both — the wrong tool for either input.

Matching the algorithm to the input's structure is the real skill this week tests, not memorizing the update loops.

Concept Thot

Four sorts, one job. Now the vocabulary that decides between them.

Concepts That Click

Section 4

Stability and the Close

Concepts That Click

Stability and In-Place, Made Concrete

- **Stable**: equal keys keep their original relative order. This matters when records are sorted by a *secondary* key after already being sorted by a primary one — a stable sort preserves the earlier ordering within ties.

Worked example — stability

Sort `[(A,2), (B,1), (C,2)]` by the number. A stable sort yields `[(B,1), (A,2), (C,2)]` — `A` stays before `C` (both key 2). An unstable sort may return `[(B,1), (C,2), (A,2)]` — same keys, different record order.

- **In-place**: $O(1)$ auxiliary space beyond the input. Insertion, selection, and bubble sort are in-place; counting sort needs a $\Theta(n+k)$ output buffer and a $\Theta(k)$ tally, so it is not.
- Stable: insertion, bubble, counting. Not stable: selection (its long-range swap can leap an equal key).

Three Classic Mistakes

- Binary searching unsorted data — no crash, just a plausible wrong answer. The precondition is a contract: sort first, or use linear search.
- Forgetting the overflow in `mid` — `(lo + hi) / 2` overflows `int` for large indices; write `lo + (hi - lo) / 2`. The same trap as `mid * mid` in `isqrt` and `(long long)` casts in C.
- Claiming a sort is $\Theta(n \log n)$ "because it sorts" — the elementary sorts are all $\Theta(n^2)$ worst case. State the *case* and the *bound*, and check whether the algorithm even compares keys (counting sort does not).

How to revise

For each sort, write one sentence: the invariant, the best case, the worst case, and whether it is stable. Then trace one pass by hand. The test is traces and cases, not definitions.

Week 8 Summary

- **The search contract**: linear search needs no order, $\Theta(n)$ worst case; binary search *requires* sorted input and is $\Theta(\log n)$ worst case — but silently wrong on unsorted data.
- **Binary search on a monotone predicate** answers questions about values that are not stored: $\lfloor \sqrt{n} \rfloor$ (search the answer space $0..n$), and the longest common prefix (search the prefix length; sort once, then compare first and last).
- **Elementary sorts**: insertion (adaptive, stable, $O(n)$ best); selection (always $\Theta(n^2)$, $n-1$ swaps, unstable); bubble (stable, adaptive with an early exit); counting ($\Theta(n+k)$, stable, $O(n+k)$ space, small integer keys).
- **Dictionary at the end**: stable, in-place, best/worst case — the three properties that decide which sort fits an input.

Next week

Why can no *comparison* sort beat $\Theta(n \log n)$, and which algorithms escape that bound? Efficient sorting: quick, merge, shell, radix, heap.

Before You Go: Predict

Three one-line predictions

Answer from the algorithms, then check in the lab.

1. Binary search on a sorted array of $1{,}000$ elements: about how many comparisons in the worst case, and how many would linear search need?
2. Sorting `[3, 1, 3, 2]` — which of insertion, selection, bubble, counting is stable, and what order do the three 3s/ties take?
3. Is the "first occurrence" variant of binary search different from ordinary binary search? Why return a candidate `mid` instead of discarding it?

Answers: ≈ 10 ($\log_2 1000$) versus up to $1{,}000$ — order is worth roughly a hundredfold here. Insertion, bubble, and counting are stable; selection may reorder equal keys. Yes, it differs: keeping `mid` as a candidate when `A[mid] == key` lets the search continue left to find the *first* match.

References I


Ellis Horowitz and Sartaj Sahni.
Fundamentals of Data Structures.
Galgotia Booksource, 2nd edition, 2008.


Narasimha Karumanchi.
Data Structures And Algorithms Made Easy.
CareerMonk Publications, 2017.
This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.


Donald E. Knuth.
The Art of Computer Programming, Volume 3: Sorting and Searching.
Addison-Wesley, 2nd edition, 1998.


Robert Sedgewick and Kevin Wayne.
Algorithms.
Addison-Wesley, 4th edition, 2011.

Concepts That Click

References II


Niklaus Wirth.
Algorithms and Data Structures.
Author (freely distributed), 2004.
Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.

Concepts That Click